Let's see how `SGen` affects the definition of `Prop` and `Prop.forAll`. The `SGen` version of `forAll` looks like this:

```
def forAll[A](g: SGen[A])(f: A => Boolean): Prop
```

Can you see why it's not possible to implement this function? `SGen` is expecting to be told a size, but `Prop` doesn't receive any size information. Much like we did with the source of randomness and number of test cases, we simply need to add this as a dependency to `Prop`. But since we want to put `Prop` in charge of invoking the underlying generators with various sizes, we'll have `Prop` accept a *maximum* size. `Prop` will then generate test cases up to and including the maximum specified size. This will also allow it to search for the smallest failing test case. Let's see how this works out.[6]

Listing 8.4 Generating test cases up to a given maximum size

```
type MaxSize = Int
case class Prop(run: (MaxSize,TestCases,RNG) => Result)

def forAll[A](g: SGen[A])(f: A => Boolean): Prop =
  forAll(g(_))(f)

def forAll[A](g: Int => Gen[A])(f: A => Boolean): Prop = Prop {
  (max,n,rng) =>
    val casesPerSize = (n + (max - 1)) / max
    val props: Stream[Prop] =
      Stream.from(0).take((n min max) + 1).map(i => forAll(g(i))(f))
    val prop: Prop =
      props.map(p => Prop { (max, _, rng) =>
        p.run(max, casesPerSize, rng)
      }).toList.reduce(_ && _)
    prop.run(max,n,rng)
}
```

For each size, generate this many random cases.

Make one property per size, but no more than `n` properties.

Combine them all into one property.

8.4 Using the library and improving its usability

We've converged on what seems like a reasonable API. We could keep tinkering with it, but at this point let's try *using* the library to construct tests and see if we notice any deficiencies, either in what it can express or in its general usability. Usability is somewhat subjective, but we generally like to have convenient syntax and appropriate helper functions for common usage patterns. We aren't necessarily aiming to make the library more expressive, but we want to make it pleasant to use.

[6] This rather simplistic implementation gives an equal number of test cases to each size being generated, and increases the size by `1` starting from `0`. We could imagine a more sophisticated implementation that does something more like a binary search for a failing test case size—starting with sizes `0,1,2,4,8,16...`, and then narrowing the search space in the event of a failure.